show the form back to the user. To do this, change the `new` and `create` actions inside `app/controllers/articles_controller.rb` to these:

```
def new
  @article = Article.new
end

def create
  @article = Article.new(article_params)

  if @article.save
    redirect_to @article
  else
    render 'new'
  end
end

private
  def article_params
    params.require(:article).permit(:title, :text)
  end
```

The `new` action is now creating a new instance variable called `@article`, and you'll see why that is in just a few moments.

Notice that inside the `create` action we use `render` instead of `redirect_to` when `save` returns `false`. The `render` method is used so that the `@article` object is passed back to the `new` template when it is rendered. This rendering is done within the same request as the form submission, whereas the `redirect_to` will tell the browser to issue another request.

If you reload http://localhost:3000/articles/new and try to save an article without a title, Rails will send you back to the form, but that's not very useful. You need to tell the user that something went wrong. To do that, you'll modify `app/views/articles/new.html.erb` to check for error messages:

```
<%= form_for :article, url: articles_path do |f| %>

  <% if @article.errors.any? %>
    <div id="error_explanation">
      <h2>
        <%= pluralize(@article.errors.count, "error") %> prohibited
        this article from being saved:
      </h2>
      <ul>
        <% @article.errors.full_messages.each do |msg| %>
          <li><%= msg %></li>
        <% end %>
      </ul>
    </div>
  <% end %>

  <p>
    <%= f.label :title %><br>
    <%= f.text_field :title %>
  </p>

  <p>
    <%= f.label :text %><br>
    <%= f.text_area :text %>
  </p>

  <p>
    <%= f.submit %>
  </p>

<% end %>
```